

Analysis Of Algorithms

Name: MAJD A. H. ALZUHRI

SAP ID: 71591

DATE:04/10/2026

GitHub Repository Link: <https://github.com/MajdAlzuhri/Fall26AoA2026>

Q1. Algorithm Design and Asymptotic Analysis

Given an array of n integers, design an algorithm to find the largest and smallest elements in the array.

1. Provide a clear explanation of the algorithmic strategy used.
2. Write the complete pseudocode of your proposed algorithm using the notation discussed in class.
3. Identify the major primitive operations performed by your algorithm.
4. Analyze the best-case and worst-case running time of your algorithm.
5. Express the running time using appropriate asymptotic notation.
6. Determine the space complexity of your algorithm.

Q1.Sloution:

Day: _____

Date: _____

Name: MAJID A. H. ALZUHRI

SAP ID: 71591

Assignment - 01

Q1) Algorithm Design and Asymptotic Analysis

1) Algorithmic Strategy

To find the smallest and largest element, I will scan the array once.

First, I will assign the first element to both smallest and largest. Then, I will compare each remaining element with these two values.

If an element smaller than smallest, I will update smallest. If it is larger than largest, I will update largest.

Assumption: The array contains at least one element, so $n \geq 1$. The indexes 0 to $n-1$.

sap id: 71591

(2)

Day: _____

Date: _____

2. Pseudocode

Algorithm Find-Min-Max(A, n)

[Find the smallest and largest elements in array A]

Step 1 [Initialize the two values]

smallest = A[0]

Largest = A[0]

Step 2 [Check the remaining elements]

For i = 1 To n - 1

if A[i] < smallest Then

smallest = A[i]

End if

if A[i] > Largest Then

Largest = A[i]

End if

End For

Step 3 [Return the result]

Return (smallest, Largest)

Sap id: 71591

(3)

Day: _____

Date: _____

4. Major primitive Operations

Operation

Example

Array access

$A[i]$

Comparison

$A[i] < \text{smallest}$

Assignment

$\text{smallest} = A[i]$

Loop comparison

Checking whether the
loop should continue

Counter increment

Increasing i by 1

Return

Return smallest and
Largest

5. Best-Case Running Time

The best case occurs when no updates are needed after initialization, for example all elements have the same value.

However, the algorithm still checks all remaining elements.

The loop runs: $n - 1$ times

Each iteration performs two elements comparison

$$C_{\text{best}}(n) = 2(n - 1) = 2n - 2$$

Sap id: 71591

19/11/20 (4)

Day: _____

Date: _____

Therefore, the best case running time is:

$$T_{\text{best}}(n) = O(n)$$

6. Worst Case Running Time

In worst case, many elements cause updates to Smallest and Largest.

The algorithm still performs two elements comparison for each of the remaining $n-1$ elements:

$$C_{\text{worst}}(n) = 2(n-1) = 2n - 2$$

Each iteration takes constant time, including any updates, Therefore:

$$T_{\text{worst}}(n) = O(n)$$

The number of assignments may change with the input order, but the growth rate remains linear.

Sap id: 71591

(5)

Day: _____

Date: _____

7. Asymptotic Notation

For both the best and worst case:

Notation	Bound
Big-O	$O(n)$
Big- Ω	$\Omega(n)$
Big- Θ	$\Theta(n)$

The tight bound is $\Theta(n)$ because the algorithm must examine every element.

8. Space Complexity

The algorithm use only a fixed number of extra variables:

Smallest, Largest, i

It does not create another array.

Therefore: Auxiliary Space = $\Theta(1)$

If the input array is included, the space is:

Total Space = $\Theta(n)$

Q2. Design and Compare Algorithms

Given n integers, design an algorithm to determine whether a particular value X exists in the array.

1. Design two different algorithms/strategies to solve the problem.
2. Write the pseudocode for both algorithms.
3. Analyze the best-case and worst-case time complexity of each algorithm.
4. Express their complexities using Big-O, Big- Ω , and Big- Θ , wherever applicable.
5. Compare the two algorithms and justify which algorithm would be more efficient for different input conditions.
6. Explain how your choice of algorithm is affected by the size and arrangement of the input data.

Q2.Sloution:

Sap id: 71591

PRUF: 6 (6)

Day: _____

Date: _____

Q2. Design and Compare Algorithms

1. Problem

Array of n integers and value X ,
determine whether X exists in the array.

I will use two strategies:

1. Linear Search.
2. Binary Search.

Both algorithms return true if the value exists and false otherwise.

2. Linear Search

Strategy: Check the elements one by one.
~~The~~ The array does not ~~to~~ need to be stored.

Algorithm Linear-Search(A, n, X)

{check whether X exists in A }

Step 1 [check each element]

For $i = 0$ To $n - 1$

 If $A[i] = X$ Then

 Return true

 End If

End For

Sap id: 71591

IP2/F:4 (7)

Day: _____

Date: _____

Step 2 [Value not found]
Return false

Time Analysis:

⇒ Best case: X is first element. One comparison is needed, so the time is $O(1)$.

⇒ Worst case: X is last or absent. All n elements are checked, so time is $O(n)$.

2. Binary Search

Strategy: Check the middle element.
If it equals X , return. Otherwise, continue in the left or right half.

Precondition: The array must be stored in ascending order

Algorithm Binary-Search(A, n, X)

[check whether X exists in a sorted array]

Step 1 [Set the boundaries]

Low $= 0$

High $= n - 1$

sap id: 71591

(8)

Day: _____

Date: _____

Step 2 [Search the remaining range]

while Low $\leq$ High
Mid = Low + floor((High - Low) / 2)

If A[Mid] = X then
Return True
End If

If X < A[Mid] Then
High = Mid - 1
Else
Low = Mid + 1
End If
End while

Step 3 [Value not found]
Return False

Time Analysis

⇒ Best case: X is the first middle element, so the time is $\Theta(1)$.

⇒ Worst case: The search range is halved each iteration:

$$\frac{n}{2^k} = 1 \Rightarrow k = \log_2 n$$

Therefore, the worst case time is

$$\Theta(\log n).$$

Day:

Date:

3. Asymptotic Comparison

Algorithm	Case	Big-O	Big- Ω	Big- Θ
Linear Search	Best	$O(1)$	$\Omega(1)$	$\Theta(1)$
Linear Search	Worst	$O(n)$	$\Omega(n)$	$\Theta(n)$
Binary Search	Best	$O(1)$	$\Omega(1)$	$\Theta(1)$
Binary Search	Worst	$O(\log n)$	$\Omega(\log n)$	$\Theta(\log n)$

4. Comparison and Choice

⇒ Linear Search works on any array and is suitable for small arrays or one search in unsorted data.

⇒ ~~Binary Search works on any array and~~

⇒ Binary Search is usually more efficient for large, sorted arrays, because it removes half of the search range each iteration.

⇒ If the array is unsorted, sorting must be included in the cost, Using $\Theta(n \log n)$ sort followed by one Binary Search gives a total of $\Theta(n \log n)$. Therefore, Linear Search is usually better for single search in unsorted data.

